

CSC 4553

Fall 2026 - Day 6

Admin

- Lab 3 due Friday, 18:00
- Quiz 3 due Monday, 18:00
- Programming assignment 1 due Wednesday, 18:00

Process management

Questions to consider

- Which system calls are related to process management and lifecycles?
- How does the process hierarchy work?
- What are zombies and orphans? Why do zombies exist?

UNIX process APIs

- `fork()` creates a new child process
 - All processes are created by forking from a parent
 - The `init` process is ancestor of all processes
 - Run `ps tree` in a terminal to see
- `exec()` makes a process execute a given executable (effectively replaces the process)
- `exit()` terminates a process
- `wait()` causes a parent to block until child terminates
- Many variants exist of the above system calls with different arguments

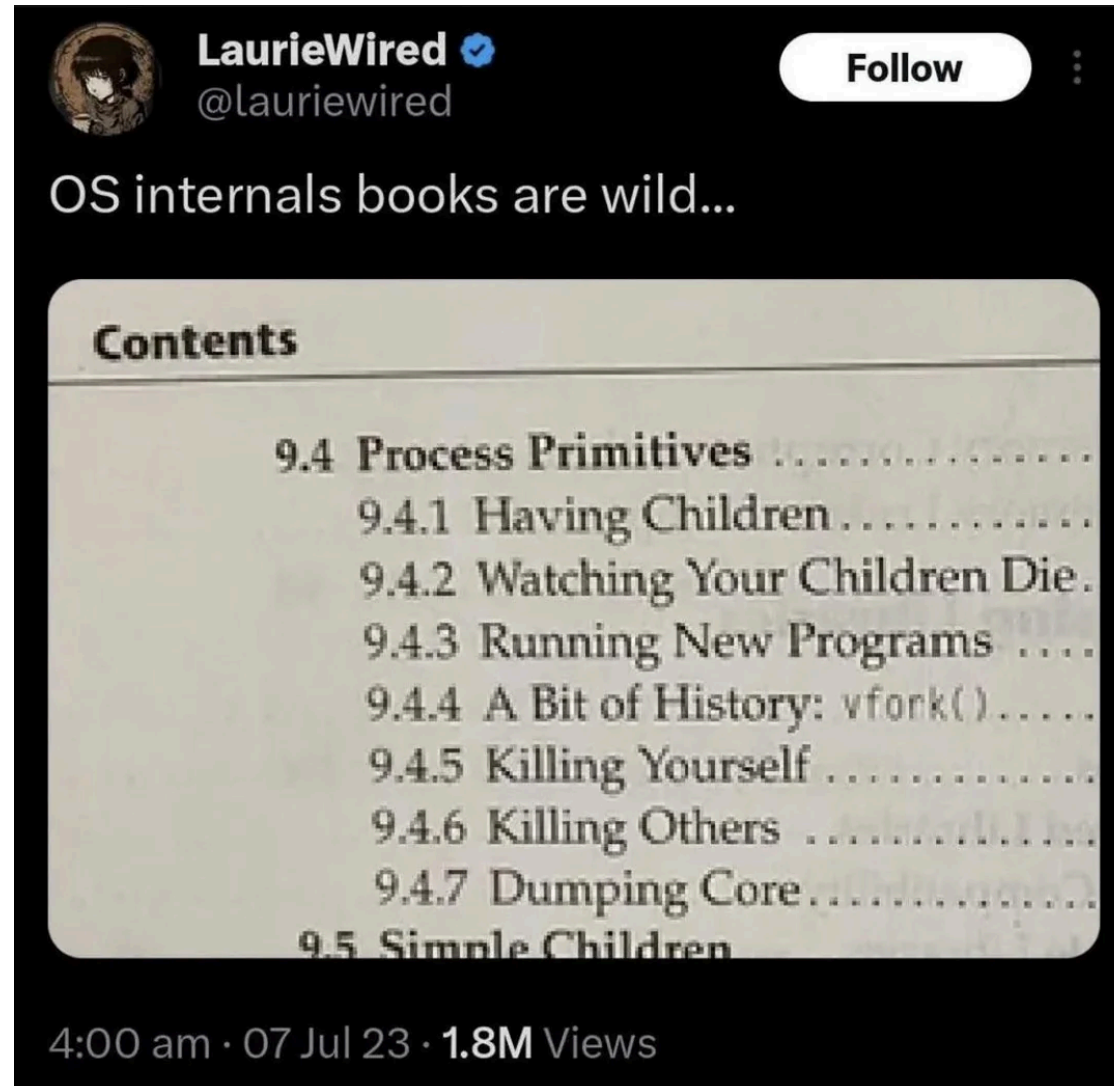
What happens during a `fork()`?

- A new process is created by making a copy of parent's memory image
- Both parent and child have unique address spaces (isolated from each other, allowing for independent processing)
- The new process is added to the OS process list and scheduled
- Parent and child start execution just after fork (with different return values)
- Parent and child execute and modify the memory data independently

Process creation

- Different execution models
 - Parent & child may execute independently
 - Parent may wait for child
 - Child may create more children (Process hierarchies)
 - Parent may kill children
- Child often invokes `exec()` to change its memory image to a new program
- Why two steps (`fork()` then `exec()`)?
 - Allows the child to change file descriptors and other settings before `exec()`

Process destruction



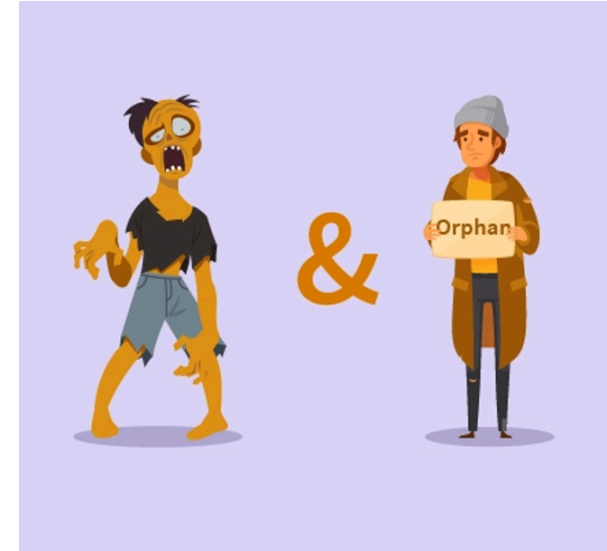
Process destruction (cont'd)

- Some operating systems do not allow child to exist if its parent has terminated. If a process terminates, then all its children must also be terminated
 - **Cascading termination**: All children, grandchildren, etc., are terminated.
 - The termination is initiated by the operating system
- The parent process may wait for termination of a child process by using the `wait()` system call. The call returns status information and the pid of the terminated process

```
1 pid = wait(&status);
```

Zombies and orphans

- If no parent waiting (did not invoke `wait()`), and process completes, process is a **zombie**
 - Zombie = dead but not yet reaped (exit status hasn't been read)
 - Still has an entry in the process table
 - We need zombies: so the kernel can preserve a child's exit status until the parent calls `wait()`, even if the child exits first
- If parent terminated without invoking `wait()`, process is an **orphan**
 - Orphan = alive but parent is gone
 - `init` benevolently adopts orphans



What isn't clear?

Comments? Thoughts?

Modern process isolation

Questions to consider

- What are namespaces and cgroups?
- How do they differ from virtual machines?
- How do containers use them to isolate processes?

Isolation without full virtualization

- **Virtual machines** provide complete isolation by emulating entire hardware + OS
 - Heavier resource overhead (multiple OS instances)
 - Better isolation between workloads
- **Containers** provide lightweight isolation using OS-level mechanisms
 - Share the same kernel
 - **Much lower overhead than VMs**
 - Linux kernel provides the building blocks: **namespaces** and **cgroups**

Namespaces: logical isolation (visibility)

- Namespaces partition global system resources so they appear as separate isolated instances
- Each process belongs to a namespace and only sees resources in that namespace
- Types of namespaces:
 - **PID**: process IDs (what processes can a process see?)
 - **Network**: network interfaces, ports, routing tables
 - **Mount**: filesystem mounts (what can a process access?)
 - **IPC**: IPC objects, message queues
 - **UTS**: hostname and domain name
 - **User**: user and group IDs (who owns the process?)

Namespaces example

- Two processes in separate PID namespaces think they are PID 1 (init)
- Each sees only processes within their own namespace
- From the host OS perspective, they have different global PIDs
- Enables the illusion that each container has its own isolated process tree

Seeing namespaces in action

- View namespaces a process belongs to:

```
1 ls -l /proc/self/ns/
```

- Use **unshare** to create a new PID namespace:

```
1 unshare -pf --mount-proc bash      # creates new PID namespace, your shell is PID 1
2 ps aux                             # only sees processes in this namespace
3 exit                               # back to host namespace
4 ps aux                             # will show all processes on the host
```

- Compare namespace inodes before and after (same inode = same namespace):

```
1 ls -i /proc/self/ns/pid
2 unshare -pf --mount-proc bash -c 'ls -i /proc/self/ns/pid' # different inode
```

Cgroups: resource limits

- **cgroups (control groups)** limit, prioritize, and account for resource usage of process groups
- Key capabilities:
 - **CPU limits**: restrict how much CPU time a group can use
 - **Memory limits**: cap memory usage; OOM killer invoked if exceeded
 - **I/O limits**: restrict disk I/O bandwidth
 - **Device access**: restrict which devices a process can access
- All processes in a cgroup share the same resource limitations

Seeing cgroups in action

- View what cgroup a process belongs to:

```
1 cat /proc/self/cgroup
```

- Check your current limits:

```
1 cat /proc/self/limits # shows per-process limits (some enforced by cgroups)
```

- In practice, cgroups are invisible to users, kernel enforces limits automatically when a process exceeds allocated resources

Cgroups vs. namespaces

- **Namespaces**: about **visibility**—what can a process see?
 - Logical isolation of resources
- **cgroups**: about **limits**—how much can a process use?
 - Resource accounting and enforcement
- Together: processes appear isolated AND are prevented from consuming excessive resources

Containers as a building block

- cgroups and namespaces are **mechanisms**, containers allow us to apply **policy** through orchestration
- Containers (e.g. **Docker**) allow for **policy**: that can be orchestrated by combining namespaces + cgroups + layered filesystems
- Results in lightweight, portable process isolation
- Single kernel, multiple isolated environments
- Much cheaper than VMs, **but** with worse isolation guarantees

Namespace or cgroup?

- **Think (2 min)**: for each symptom, is the mechanism you need a **namespace**, a **cgroup**, or both?
 1. A process in one container runs **ps aux** and sees another container's processes
 2. One container's runaway loop pegs all 8 cores; everything else crawls
 3. Two containers each want to bind port 80
 4. A container can read **/etc/shadow** from the host filesystem
 5. A container allocates until the host itself starts swapping
 6. A process is PID 1 to itself but PID 4242 to the host
- **Discuss (2 min)**: compare. Any you'd label "both?"
- **Share (1 min)**: which of these is a **security** boundary? If containers do all of this, why do cloud providers still run untrusted code in VMs?

What isn't clear?

Comments? Thoughts?